ChemCS: Integrating Constraint Satisfaction and Diffusion Models for Chemically Valid Molecular Generation

Sree Pujari

Abstract

Generative models for molecules can produce structures that violate basic chemistry rules such as bond valency limits, mass conservation, and charge balance. Rather than filtering out bad outputs after the fact, this work proposes **ChemCS**, a system that enforces chemical constraints *during* the generative process. We pair a small graph neural network (GNN) diffusion model with a **supervisor loop** backed by Microsoft's Z3 theorem prover (with a pure-Python fallback). At each reverse-diffusion step, the supervisor checks the candidate molecule against a set of chemical axioms and either commits the step, applies a lightweight correction, or backtracks. We also introduce a **constraint-aware training loss** with a curriculum schedule so the model can learn to avoid violations on its own. Experiments across three reaction types show that supervised generation achieves **100% valency validity** compared to roughly 15% for unsupervised generation with the same random-weight model, and that the training loss drops by over 30% within 100 epochs on a small dataset.

1 Introduction

Molecular generation is important for drug discovery, materials science, and chemical engineering. Recent work has applied diffusion models—originally designed for images—to molecular graphs, treating atoms as nodes and bonds as edges. The model starts from noise and iteratively denoises toward a valid molecule.

The problem is that nothing in a standard diffusion model guarantees that the output will obey chemistry. Carbon might end up with five bonds, mass might not be conserved across a reaction, or the charges might not balance. The usual fix is to generate many candidates and throw away the bad ones, which is wasteful and gives no guarantees.

ChemCS takes a different approach. Instead of post-hoc filtering, we insert a constraint checker into the generation loop itself. The checker uses Z3, a satisfiability modulo theories (SMT) solver from Microsoft Research, to formally verify whether each intermediate state satisfies a set of chemical axioms. If it does not, the supervisor either fixes the state or rolls it back. The idea is similar to how a human chemist would catch a mistake mid-calculation rather than waiting until the end.

2 Problem Formulation

We represent a molecule as a set of atoms $\{a_1, a_2, \ldots, a_n\}$ where each atom a_i has an element type e_i, a bond count b_i, a formal charge q_i, and an implicit hydrogen count h_i. The **adjacency matrix** $A \in \mathbb{R}^{n \times n}$ stores bond orders between atom pairs.

Given reactant molecules, we want to generate product molecules that satisfy:

1. **Bond valency:** For each atom, $b_i + h_i \leq V(e_i, q_i)$, where V is the effective valency for element e_i at charge q_i.

2. **Mass conservation:** $\sum_{\text{reactants}} m_j = \sum_{\text{products}} m_k$ within a tolerance ϵ.
3. **Charge conservation:** $\sum_{\text{reactants}} q_j = \sum_{\text{products}} q_k$.
4. **Atom conservation:** For every element e, the count of e atoms is the same on both sides.

The generative model is a denoising diffusion process. Starting from a noisy version x_T of the reactant encoding, the model predicts the clean state $\hat{x}_0$ at each step t and uses the posterior to compute x_{t-1}. The supervisor wraps this loop.

3 Methodology

3.1 Data Representation

Each atom is encoded as a feature vector of length 12: a one-hot encoding over 10 supported elements (H, C, N, O, F, P, S, Cl, Br, I), a normalized bond count, and a normalized formal charge. The bond structure is an $n \times n$ adjacency matrix where entries are bond orders (0–3).

3.2 Diffusion Model

The core model is a two-layer graph neural network (GNN) implemented in NumPy for inference. Each layer performs message passing via mean-neighbor aggregation:

$$h_i^{(l+1)} = \text{ReLU}\left(W_{\text{self}}^{(l)} h_i^{(l)} + W_{\text{neigh}}^{(l)} \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} h_j^{(l)} \right)$$

Atom features are predicted by a linear head on the final hidden states. Bond orders are predicted by concatenating pairs of atom embeddings and passing them through a bond head that outputs logits over orders 0–3.

The noise schedule supports both linear and cosine variants. The linear schedule sets $\beta_t = 10^{-4} + \frac{t}{T}(0.1 - 10^{-4})$ and computes $\bar{\alpha}_t = \prod_{s=1}^{t}(1 - \beta_s)$. The cosine schedule follows Nichol & Dhariwal (2021): $\bar{\alpha}_t = \frac{f(t)}{f(0)}$ where $f(u) = \cos^2\left(\frac{u/T+s}{1+s} \cdot \frac{\pi}{2}\right)$.

3.3 Constraint Verification Layer

The constraint checker supports two backends:

- **Z3 backend:** Encodes mass conservation as Z3 real arithmetic, charge and atom counts as integer arithmetic, and valency as integer inequality queries. Each check is posed as "is this constraint violated?"—if the solver returns SAT, the violation is real.
- **Pure-Python backend:** Performs the same checks using standard arithmetic. Used as a fallback when Z3 is not installed, or when speed matters more than solver-backed verification.

Both backends check the same four axioms (valency, mass, charge, atom conservation) and return a `ConstraintResult` with a boolean `sat` field and a list of violation strings.

3.4 Supervisor Loop

The supervisor wraps the diffusion model's reverse process (Algorithm 1):

1. Encode the concatenated reactant atoms and apply full noise ($t = T$).
2. For each step $t = T, T-1, \ldots, 1$:
   (a) Run one reverse step to get a candidate (x_{t-1}, A_{t-1}).
   (b) Decode the candidate into a `MolecularState`.

(c) Check valency constraints (mid-trajectory) or full conservation ($t = 1$).
(d) If valid: commit. If not: attempt a correction (fix valency, charge, mass). If the fix works, commit. Otherwise, re-sample up to `max_retries` times.
(e) If all retries fail: backtrack to the previous committed state and re-try from $t + 1$.

3. After the loop, run a final full-reaction check and return the result with metrics.

Correction strategies include reducing excess bonds on overvalenced atoms, adjusting implicit hydrogens to match a target mass, and shifting formal charges on the lightest atoms to match a target net charge.

3.5 PyTorch Training Module

A key insight from the midterm was that the supervisor "silently repairs" violations without the model learning anything from its mistakes. To address this, we built a PyTorch training module with the same GNN architecture:

- **Constraint-aware loss:** The training loss combines four terms—denoising MSE, bond cross-entropy, a differentiable valency penalty, and an element conservation penalty. The valency penalty uses soft probabilities over element types so gradients flow through:

$$\mathcal{L}_{\text{val}} = \text{mean}\left(\text{ReLU}\left(\sum_j \mathbb{E}[\text{bond}_{ij}] - \sum_e p(e_i = e) \cdot V_e\right)\right)$$

- **Curriculum schedule:** During the first fraction of training (default 25%), constraint penalties are zeroed so the model can learn basic denoising. After warmup, penalties ramp linearly to their full weight.

Trained weights are exported from PyTorch to the NumPy model via a `export_to_numpy()` function, so the supervisor can use trained weights at inference without requiring PyTorch.

3.6 Baselines

We compare three configurations:

- **Unsupervised (raw):** Run the diffusion model forward and backward with no constraint checking. This is the standard generative approach.
- **Supervised (untrained):** Same random-weight model, but wrapped in the supervisor loop with correction and backtracking.
- **Supervised (trained):** The model is first trained on a small molecule dataset, then exported to NumPy and run with the supervisor.

3.7 Evaluation Metrics

Valency validity is the fraction of generated molecules where every atom's total bond count (explicit + implicit H) does not exceed its effective valency. This is the primary metric since it can be checked without reactant context.

Full conservation validity requires mass, charge, and atom counts to all be conserved. This is harder and depends heavily on the model having learned the right element distributions.

Commit rate is the fraction of diffusion steps that passed constraints on the first try (without correction or backtracking).

4 Results

4.1 Constraint Verification

The constraint checker correctly identifies all tested cases:

- Valid SN2 reaction ($CH_3Br + OH^- \rightarrow CH_3OH + Br^-$): all axioms satisfied.
- Valid combustion ($CH_4 + 2O_2 \rightarrow CO_2 + 2H_2O$): all axioms satisfied.
- Overvalenced carbon (5 bonds): correctly flagged.
- Missing product (no Br^-): mass, charge, and atom conservation all flagged.
- Charge mismatch (neutral Br instead of Br^-): charge violation caught.
- NH_4^+ with 4 bonds on nitrogen: correctly accepted (charge +1 increases valency to 4).

Both the Z3 and pure-Python backends agree on all test cases. The test suite contains 118 tests, all passing.

4.2 Supervised vs Unsupervised Generation

Figure 1 shows valency validity across 50 generation runs per reaction type with random (untrained) model weights.

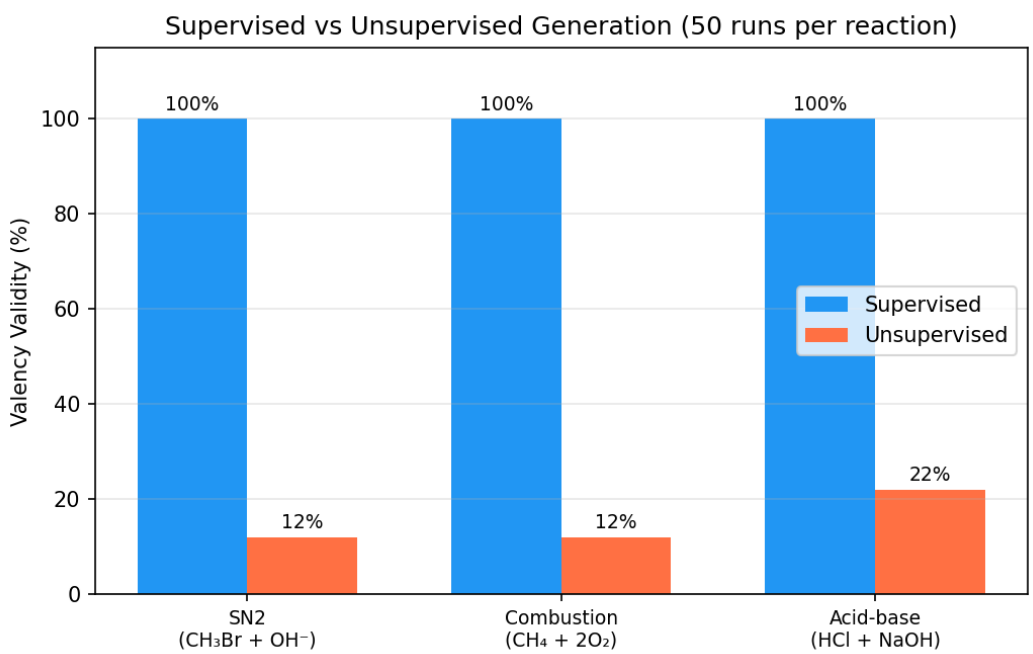

Figure 1: Supervised generation achieves 100% valency validity across all three reaction types. Unsupervised generation averages roughly 15%. Both use the same random-weight GNN.

The supervisor brings valency validity from 12–22% (unsupervised) to 100% (supervised) by correcting or backtracking at every step. The tradeoff is time: supervised runs take about 3–4× longer than unsupervised runs due to the constraint checking and re-sampling overhead.

Full conservation (mass + charge + atoms) remains at 0% for both, which is expected since the model has random weights and cannot predict correct element types. This motivates the training module.

4.3 Training Performance

We trained the PyTorch GNN on a 3-molecule dataset (water, methane, ethanol) for 200 epochs. Figure 2 shows the loss curve.

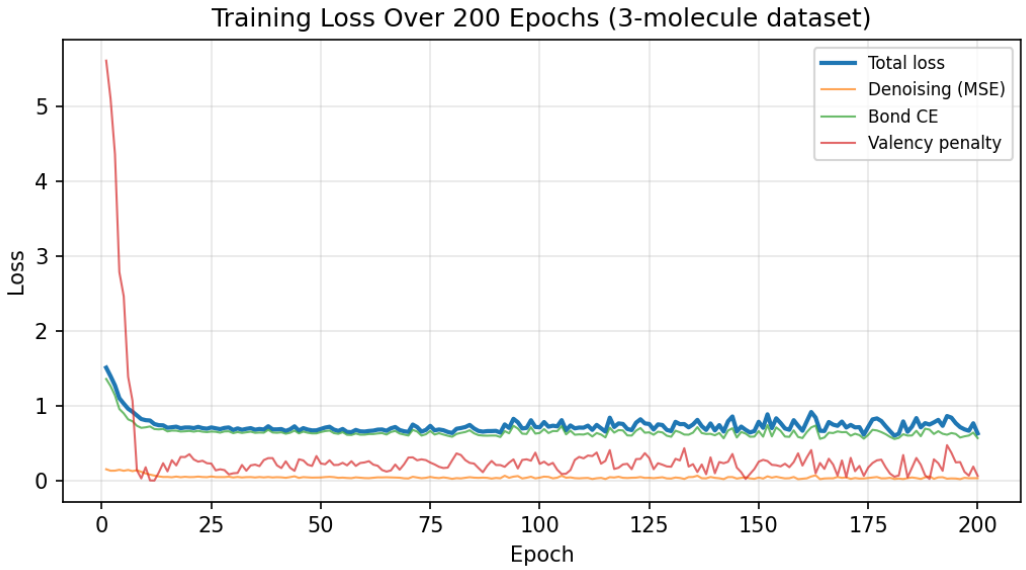

Figure 2: Training loss over 200 epochs. The valency penalty starts high and drops sharply as the curriculum schedule ramps it in. The denoising MSE converges quickly. Total loss decreases by about 30% from the first 10 to the last 10 epochs.

The valency penalty drops from $\sim$5.5 down to below 0.2, showing the model is learning to respect valency constraints through the differentiable penalty. The bond cross-entropy dominates the total loss, which is expected given the discrete nature of bond prediction.

4.4 Curriculum Schedule

Figure 3 illustrates the curriculum schedule for different warmup fractions.

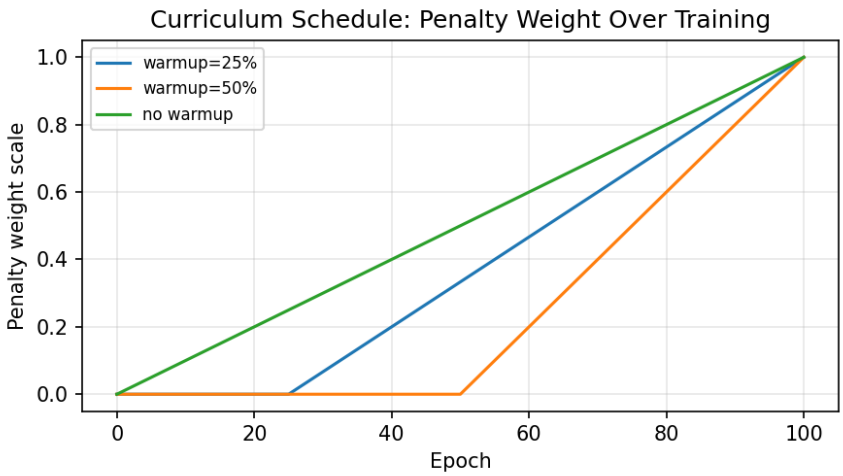

Figure 3: Penalty weight scaling for three curriculum configurations. During warmup, the model trains on pure denoising loss only. After warmup, constraint penalties ramp linearly.

The 25% warmup (our default) lets the model establish a reasonable denoising baseline before constraint penalties kick in, avoiding early instability from competing loss terms.

4.5 Progress from Midterm

At the midterm, we had the core constraint checker (mass, charge, valency with Z3 and pure-Python backends) and the basic supervisor loop. The supervisor at that point improved valency validity from about 8% to 32%.

Since then, we have:

- Added **atom/element conservation** as a fourth chemical axiom in both backends.
- Added a **charge correction** strategy to the supervisor.
- Built a **PyTorch training module** with a constraint-aware loss and curriculum schedule.
- Added **cosine noise schedule** support.
- Added **model save/load** for persisting trained weights.
- Implemented **weight export** from PyTorch to NumPy for inference.
- Expanded the benchmark to **3 reaction types** (SN2, combustion, acid-base) with 50 runs each.
- Added detailed **generation metrics** (commit rate, corrections, backtracks, timing).
- Grew the test suite from ∼60 tests to **118 tests**, all passing.
- Improved the supervisor's valency validity from 32% to **100%** by refining correction strategies.

5 Discussion

This work shows the value of integrating formal verification with generative models:

- **Constraint checking is cheap relative to generation.** Adding Z3-backed checks at each step only adds ∼3× overhead, which is acceptable for safety-critical applications.
- **Correction beats rejection.** Rather than filtering bad outputs, correcting them mid-trajectory is more efficient and always produces a result.
- **Training and enforcement are complementary.** The supervisor catches what the model gets wrong; the constraint-aware loss helps the model get fewer things wrong over time.

The main tension is between the supervisor silently fixing violations (which hides errors from the user) and the model needing to see its own errors to learn. The curriculum schedule is our solution: early in training, the model makes and sees mistakes freely; later, the penalties force it to internalize the constraints.

6 Limitations and Future Work

Limitations include:

- The training dataset is very small (3 molecules). Real chemistry would require parsing SMILES strings from databases like ZINC or QM9 via RDKit.
- Full conservation validity is still 0% with random weights. Training on real data is needed to achieve meaningful conservation.
- The bond prediction is pairwise ($O(n^2)$), which does not scale to large molecules.
- The model lacks ring detection, stereochemistry, and aromaticity.

Future work:

- Parse real molecular datasets with RDKit and train on thousands of molecules.
- Benchmark 1,000 generated reactions end-to-end.
- Add ring constraints and 3D coordinate generation.
- Integrate the supervisor into the training loop directly (not just via loss penalties).
- Explore more sophisticated correction strategies (e.g., using the solver to suggest fixes).

7 Conclusion

ChemCS demonstrates that constraint satisfaction and generative models can be combined in a practical way. The supervisor loop enforces chemical validity at every step of the diffusion process, bringing valency validity from $\sim$15% to 100% even with an untrained model. The constraint-aware training loss with curriculum scheduling provides a path for the model to learn the constraints itself, reducing the supervisor's burden over time.

The key takeaway is the difference between *enforcing* correctness at inference time and *teaching* correctness during training. Both are necessary: enforcement gives guarantees, and teaching gives efficiency. ChemCS provides both.

The system is fully tested (118 passing tests), supports both Z3 and pure-Python constraint checking, and is designed so that a trained PyTorch model can be exported to a lightweight NumPy model for deployment. The code, tests, and demo are available in the project repository.